

Unit I - Problem Solving Techniques Assessment

Class	Python Basics	Unit	UNIT-I
Topics	Functions, Data Abstraction, Scoping, Algorithmic Strategies	Total Marks	100
Question Mix	30 MCQ + 20 Fill-up + 10 Coding	Time	Teacher decides

Instructions

- Answer all questions.
- MCQ and fill-up questions are auto-evaluated after teacher review.
- Coding questions must include clean Python code with proper indentation.
- This student PDF does not contain answers.

Section A - MCQs (30 x 1 = 30 marks)

- [Functions]** A subroutine is best described as: **(1 mark)**
 - A. A small section of code used to perform a specific task
 - B. A hardware input device
 - C. A database table
 - D. A file extension
- [Functions]** In programming languages, a subroutine is also commonly called a: **(1 mark)**
 - A. Variable
 - B. Function
 - C. Compiler
 - D. Loop condition
- [Functions]** Which pair correctly describes parameters and arguments? **(1 mark)**
 - A. Parameters are values passed; arguments are variables in definition
 - B. Parameters are variables in function definition; arguments are passed values
 - C. Both are always constants
 - D. Both refer only to output values
- [Functions]** In a function specification, 'requires' usually represents: **(1 mark)**
 - A. A loop
 - B. A postcondition
 - C. A precondition
 - D. An output statement
- [Functions]** A recursive function is a function that: **(1 mark)**
 - A. Never returns a value
 - B. Calls itself
 - C. Cannot have parameters
 - D. Uses only global variables
- [Functions]** In a recursive factorial function, the condition $x == 0$ is mainly used as the: **(1 mark)**
 - A. Base case
 - B. Compiler rule
 - C. Memory leak
 - D. Syntax error
- [Functions]** The body of a function contains: **(1 mark)**
 - A. Only comments
 - B. The statements executed when the function is called
 - C. The operating system
 - D. Only the function name
- [Functions]** A function with explicit parameter type means: **(1 mark)**

- A. The parameter type is clearly specified
- B. The function has no name
- C. The function cannot be called
- D. The output is hidden

9. [Data Abstraction] Data abstraction mainly means: **(1 mark)**

- A. Showing all internal details
- B. Providing only essentials and hiding implementation details
- C. Deleting data from memory
- D. Converting code into images

10. [Data Abstraction] ADT stands for: **(1 mark)**

- A. Advanced Data Table
- B. Abstract Data Type
- C. Automatic Data Token
- D. Applied Digital Tool

11. [Data Abstraction] An Abstract Data Type defines: **(1 mark)**

- A. Only memory addresses
- B. What operations are to be performed
- C. Only hardware speed
- D. Only file size

12. [Data Abstraction] Which of the following is listed as an implementation of List ADT? **(1 mark)**

- A. Stack ADT
- B. Mouse driver
- C. Monitor
- D. Printer queue cable

13. [Data Abstraction] Constructors are functions used to: **(1 mark)**

- A. Destroy variables
- B. Build/create abstract data objects
- C. Print only numbers
- D. Measure time complexity

14. [Data Abstraction] Selectors are functions that: **(1 mark)**

- A. Retrieve information from an abstract data type
- B. Always create new files
- C. Stop recursion
- D. Change Python syntax

15. [Data Abstraction] In the city example, makecity(name, lat, lon) is a: **(1 mark)**

- A. Selector
- B. Constructor
- C. Loop
- D. Compiler

16. [Data Abstraction] A list in Python is constructed using: **(1 mark)**

- A. Curly braces only
- B. Square brackets with comma-separated expressions
- C. Only parentheses
- D. Angle brackets

17. [Scoping] Scope refers to: **(1 mark)**

- A. File compression
- B. Accessibility of variables/functions in parts of a program
- C. Changing screen brightness
- D. Database backup

18. [Scoping] The process of binding a variable with an object is called: **(1 mark)**

- A. Mapping
- B. Rendering
- C. Compiling

D. Plotting

19. [Scoping] Which symbol is used to map a variable with an object in the notes? **(1 mark)**

- A. ==
- B. :=
- C. +=
- D. !=

20. [Scoping] Namespaces are: **(1 mark)**

- A. Containers for mapping names to objects
- B. Types of printer
- C. Only image files
- D. Network cables

21. [Scoping] LEGB stands for: **(1 mark)**

- A. Local, Enclosed, Global, Built-in
- B. List, Error, Graph, Binary
- C. Loop, End, Go, Break
- D. Local, Equal, General, Boolean

22. [Scoping] Which scope does Python look in first when resolving a variable inside a function? **(1 mark)**

- A. Built-in
- B. Global
- C. Local
- D. Database

23. [Scoping] A global scope variable can be accessed: **(1 mark)**

- A. Only inside one loop
- B. Anywhere in the program according to the notes
- C. Only after compilation
- D. Only in comments

24. [Algorithmic Strategies] An algorithm is best defined as: **(1 mark)**

- A. A finite set of instructions to solve a task
- B. A type of monitor
- C. A computer virus
- D. An image format

25. [Algorithmic Strategies] An algorithm must return at least: **(1 mark)**

- A. Zero outputs
- B. One output
- C. Ten outputs
- D. Only text output

26. [Algorithmic Strategies] Finiteness means an algorithm: **(1 mark)**

- A. Runs forever
- B. Terminates after finite number of steps
- C. Needs no input
- D. Requires internet

27. [Algorithmic Strategies] Algorithm analysis estimates: **(1 mark)**

- A. Color of output
- B. Time and space complexity for varying input sizes
- C. Printer speed only
- D. Keyboard layout

28. [Algorithmic Strategies] A priori estimate means: **(1 mark)**

- A. Theoretical performance analysis before execution
- B. Running the program and measuring time
- C. Deleting memory
- D. Writing comments

29. [Algorithmic Strategies] Posteriori testing is also called: **(1 mark)**

- A. Performance measurement
- B. Data hiding
- C. Constructor
- D. Namespace mapping

30. [Algorithmic Strategies] Space factor is calculated as: **(1 mark)**

- A. Fixed part + Variable part
- B. Input - Output
- C. Code + Comment
- D. Speed / Compiler

Section B - Fill-ups (20 x 1 = 20 marks)

1. **[Functions]** A subroutine is a small section of code used to perform a specific _____. (1 mark)

Answer: _____

2. **[Functions]** In programming, a subroutine is also called a _____. (1 mark)

Answer: _____

3. **[Functions]** Variables in a function definition are called _____. (1 mark)

Answer: _____

4. **[Functions]** The values passed to a function are called _____. (1 mark)

Answer: _____

5. **[Functions]** A function that calls itself is called a _____ function. (1 mark)

Answer: _____

6. **[Data Abstraction]** ADT stands for Abstract Data _____. (1 mark)

Answer: _____

7. **[Data Abstraction]** Abstraction means providing essentials and hiding the _____. (1 mark)

Answer: _____

8. **[Data Abstraction]** Functions that build abstract data objects are called _____. (1 mark)

Answer: _____

9. **[Data Abstraction]** Functions that retrieve information from ADT objects are called _____. (1 mark)

Answer: _____

10. **[Data Abstraction]** A Python list is written using square _____. (1 mark)

Answer: _____

11. **[Scoping]** The accessibility of a variable in a program is called its _____. (1 mark)

Answer: _____

12. **[Scoping]** The duration for which a variable is alive is called its _____. (1 mark)

Answer: _____

13. **[Scoping]** The process of binding a variable with an object is called _____. (1 mark)

Answer: _____

14. **[Scoping]** LEGB begins with Local and ends with _____. (1 mark)

Answer: _____

15. **[Scoping]** A variable defined outside all functions has _____ scope. (1 mark)

Answer: _____

16. **[Algorithmic Strategies]** An algorithm must terminate after a finite number of steps; this property is called _____. (1 mark)

Answer: _____

17. **[Algorithmic Strategies]** Algorithm analysis is divided into a priori estimates and _____ testing. (1 mark)

Answer: _____

18. **[Algorithmic Strategies]** The two main factors of algorithm complexity are time factor and _____ factor. (1 mark)

Answer: _____

19. **[Algorithmic Strategies]** Algorithm instructions should be clear and _____. (1 mark)

Answer: _____

20. [Algorithmic Strategies] The tradeoff of using more memory to reduce execution time is called time-space _____. (1 mark)

Answer: _____

Section C - Simple Coding Questions (10 x 5 = 50 marks)

1. [Functions] Write a function `square(x)` that returns the square of `x`. **(5 marks)**

Starter code:

```
def square(x):  
    # write your code here  
    pass
```

Visible test cases:

Input: `square(5)`
Expected: 25

Input: `square(12)`
Expected: 144

Expected behavior:

Return `x * x` for every integer `x`.

2. [Functions] Write a recursive function `factorial(n)` that returns factorial of `n`. Assume `n >= 0`. **(5 marks)**

Starter code:

```
def factorial(n):  
    # write your code here  
    pass
```

Visible test cases:

Input: `factorial(0)`
Expected: 1

Input: `factorial(5)`
Expected: 120

Expected behavior:

Use base case `n == 0` and recursive case `n * factorial(n-1)`.

3. [Functions] Write a function `is_positive_square(x)` that returns `x*x` only when `x > 0`, otherwise returns `None`. **(5 marks)**

Starter code:

```
def is_positive_square(x):  
    # write your code here  
    pass
```

Visible test cases:

Input: `is_positive_square(4)`
Expected: 16

Input: `is_positive_square(-3)`
Expected: `None`

Expected behavior:

Return `x*x` if `x > 0` else `None`.

4. [Data Abstraction] Represent a city as a dictionary. Write `makecity(name, lat, lon)` that returns a city object. **(5 marks)**

Starter code:

```
def makecity(name, lat, lon):  
    # write your code here  
    pass
```

Visible test cases:

Input: `makecity('Chennai', 13.08, 80.27)['name']`
Expected: Chennai

Input: `makecity('Chennai', 13.08, 80.27)['lat']`
Expected: 13.08

Expected behavior:

Return a dictionary with keys `name`, `lat`, `lon`.

5. [Data Abstraction] Using the city dictionary, write `getname(city)`, `getlat(city)`, and `getlon(city)`. **(5 marks)**

Starter code:

```
def getname(city):  
    pass  
  
def getlat(city):  
    pass  
  
def getlon(city):  
    pass
```

Visible test cases:

```
Input: c={'name':'Chennai','lat':13.08,'lon':80.27}; getname(c)  
Expected: Chennai  
  
Input: getlat(c)  
Expected: 13.08
```

Expected behavior:

Return the corresponding dictionary field.

6. [Data Abstraction] Write a function `make_pair(a, b)` that returns a list containing a and b. **(5 marks)**

Starter code:

```
def make_pair(a, b):  
    # write your code here  
    pass
```

Visible test cases:

```
Input: make_pair(10, 20)  
Expected: [10, 20]  
  
Input: make_pair('x', 'y')  
Expected: ['x', 'y']
```

Expected behavior:

Return `[a, b]`.

7. [Scoping] Write a function `local_value()` that creates a local variable `a = 7` and returns it. **(5 marks)**

Starter code:

```
def local_value():  
    # write your code here  
    pass
```

Visible test cases:

```
Input: local_value()  
Expected: 7
```

Expected behavior:

Return the local variable value.

8. [Scoping] Write a function `shadow_value()` where a global variable `a=10` exists, but the function uses local `a=7` and returns local `a`. **(5 marks)**

Starter code:

```
a = 10  
  
def shadow_value():  
    # write your code here  
    pass
```

Visible test cases:

```
Input: shadow_value()  
Expected: 7  
  
Input: a  
Expected: 10
```

Expected behavior:

Inside the function assign `a=7` and return `a`. Do not modify global `a`.

9. [Algorithmic Strategies] Write a function `steps_to_square(x)` that returns a list of algorithm steps to find square of x. **(5 marks)**

Starter code:

```
def steps_to_square(x):  
    # write your code here  
    pass
```

Visible test cases:

```
Input: steps_to_square(4)  
Expected: ['get input', 'calculate square', 'display result']
```

Expected behavior:

Return exactly ['get input', 'calculate square', 'display result'].

10. [Algorithmic Strategies] Write a function `space_factor(fixed, variable)` that returns fixed + variable. **(5 marks)**

Starter code:

```
def space_factor(fixed, variable):  
    # write your code here  
    pass
```

Visible test cases:

```
Input: space_factor(10, 5)  
Expected: 15
```

```
Input: space_factor(0, 8)  
Expected: 8
```

Expected behavior:

Return fixed + variable.